

Instituto Tecnológico y de Estudios Superiores de Monterrey

Escuela de Ingeniería y Ciencias
Ingeniería en Ciencia de Datos y Matemáticas

Manual de Desarrollo

Gestor de Identidades v1.0

Especificación de Arquitectura de Software

Equipo 5 — Grupo 604

Nombre completo	Matrícula
Alejandro Rojas	A00839067
Daniel De Pool	A00839312
Luis Eduardo Martínez	A00840037
David Aguirre	A00839482
Arturo Camacho	A00838446

Profesores del bloque: Dr. Raúl Gómez Dr. Anas Wajid Dr. Alberto F. Martínez

Socio Formador: Casa Monarca: Ayuda Humanitaria y Apoyo al Migrante, A.C.

Índice

1. Descripción General	3
1.1. Propósito del Sistema	3
1.2. Alcance	3
1.3. Casos de Uso Principales	3
2. Arquitectura del Sistema	4
2.1. Tipo de Arquitectura	4
2.2. Componentes Principales	4
2.3. Diagrama General de Arquitectura	5
2.4. Arquitectura de Seguridad de Doble Llave	5
3. Stack Tecnológico	6
3.1. Lenguajes	6
3.2. Frameworks	6
3.3. Bibliotecas	6
4. Estructura del Proyecto	7
4.1. Organización de Carpetas	7
4.2. Descripción de Módulos	8
5. Configuración del Entorno	8
5.1. Requisitos de Software	8
5.2. Variables de Entorno	8
5.3. Archivo <code>.env</code> de Ejemplo	9
5.4. Archivos de Configuración Relevantes	9
6. Instalación y Ejecución	10
6.1. Pasos para Levantar el Sistema	10
6.2. Comandos Principales	11
6.3. Credenciales Iniciales	11
7. Base de Datos	11
7.1. Modelo de Datos	11
7.2. Tablas Principales	12
7.3. Relaciones	13
8. API / Interfaces	13
8.1. Módulo <code>usuarios</code> — Prefijo: <code>/usuarios/</code>	14
8.2. Módulo <code>expediente</code> — Prefijo: <code>/expediente/</code>	15
8.3. Portal Migrante — Prefijo: <code>/mi-expediente/</code>	15
8.4. Router Raíz — <code>config/urls.py</code>	16
9. Flujos y Lógica de Negocio	16
9.1. Flujo de Login y Desbloqueo Criptográfico	16
9.2. Flujo de Creación de Expediente	17
9.3. Flujo de Operación Crítica (Firma Digital)	18
9.4. Flujo de Acceso del Migrante por Folio	18
9.5. Flujo ARCO — Rectificación	19
9.6. Flujo ARCO — Cancelación (5 estados, 4 actores)	19
9.7. Cadena de Auditoría	19

10.Ejemplos de Uso	20
10.1. Cifrar y Descifrar Datos de un Expediente	20
10.2. Firma Digital y Verificación	21
10.3. Derivación de Clave para Portal del Migrante	21
10.4. Cifrado Transparente de Base de Datos	22
11.Manejo de Errores	22
11.1. Estrategia de Errores	22
11.2. Mensajes Comunes	23
11.3. Registro (Logs)	23
12.Pruebas	23
12.1. Tipos de Tests	23
12.2. Metodología de Ejecución: Playwright MCP	24
12.3. Estructura de la Carpeta <code>test_cases/</code>	25
12.4. Ejemplo de Caso de Prueba	26
13.Despliegue	26
13.1. Opción A: Gunicorn + Nginx (VPS / Servidor Dedicado)	26
13.2. Opción B: Hosting Compartido con cPanel (HostGator / Apache)	27
13.3. Entornos	28
13.4. Consideraciones Críticas	28
14.Seguridad	28
14.1. Autenticación	28
14.2. Manejo de Datos Sensibles	29
14.3. Middleware y Cabeceras de Seguridad	29
14.4. Configuración de Seguridad Recomendada para Producción	29
14.5. Control de Acceso por Roles	30
15.Guía de Contribución	30
15.1. Convenciones de Código	30
15.2. Flujo de Trabajo (Git)	30
15.3. Cómo Agregar Funcionalidades	31
16.Problemas Conocidos	31
16.1. Limitaciones del Hosting (HostGator)	31
16.2. Limitaciones Técnicas del Sistema	31
17.FAQ Técnica	32
17.1. Problemas Comunes al Levantar el Proyecto	32
17.2. Errores Típicos de Desarrollo	32
18.Referencias	33

1. Descripción General

1.1 Propósito del Sistema

Casa Monarca: Ayuda Humanitaria y Apoyo al Migrante, A.C. es una organización de la sociedad civil con sede en Monterrey, Nuevo León, que brinda atención integral a personas migrantes en situación de vulnerabilidad. En el ejercicio de esta labor, la organización recopila y almacena información de alta sensibilidad: nombres completos, nacionalidades, fechas de nacimiento, datos de contacto, historiales médicos y psicológicos, así como situaciones jurídicas activas.

El **Gestor de Identidades** es una plataforma web segura de gestión de expedientes de migrantes que implementa una **Arquitectura de Seguridad de Doble Llave**. Su propósito es garantizar que ningún dato personal de un migrante sea legible en reposo dentro de la base de datos, incluso ante un acceso no autorizado a los servidores o al respaldo de la base de datos. El sistema integra dos subsistemas complementarios: el *Gestor de Identidades* (administra las identidades criptográficas de los colaboradores de Casa Monarca) y el *Gestor de Expedientes con Acceso por Roles* (administra los expedientes cifrados y controla quién puede descifrarlos según su rol).

1.2 Alcance

El sistema cubre los siguientes objetivos funcionales:

- **Confidencialidad:** todos los datos personales de migrantes se almacenan cifrados con AES-256-EAX; la llave AES de cada expediente se distribuye cifrada con RSA-2048 exclusivamente a los roles autorizados.
- **Control de acceso basado en roles (RBAC):** ocho roles jerárquicos con control de acceso a nivel criptográfico, no solo a nivel de aplicación.
- **Integridad y firma digital:** toda modificación relevante a un expediente queda firmada criptográficamente por el responsable mediante RSA-2048 + SHA-256 (PKCS#1 v1.5).
- **Auditoría inviolable:** bitácora de eventos encadenada con SHA-256 que detecta cualquier manipulación retroactiva.
- **Derechos ARCO del migrante:** portal público para que el migrante ejerza sus derechos de Acceso, Rectificación, Cancelación y Oposición sin necesidad de cuenta de usuario.
- **Operación en red local (LAN):** los datos permanecen exclusivamente en servidores bajo control físico de Casa Monarca, sin dependencia de servicios en la nube de terceros.

Las limitantes del sistema en su versión actual son: opera exclusivamente en red local (no se incluye un dominio público con HTTPS), no implementa autenticación multifactor (MFA), y el hosting de producción está restringido a las opciones de infraestructura disponibles para Casa Monarca (servidor compartido en HostGator o VPS propio).

1.3 Casos de Uso Principales

Los casos de uso principales del sistema son los siguientes:

- Un **Administrador** crea un nuevo usuario con identidad criptográfica completa (par RSA-2048, certificado X.509, archivo `.key`) y le asigna un rol.

- Un **Operativo** registra un nuevo expediente de migrante; los datos se cifran con AES-256-EAX y la llave se distribuye a los roles autorizados.
- Un **Coordinador** verifica y firma digitalmente un expediente usando su archivo `.key` (análogo a la e.firma del SAT).
- Un **migrante** accede al portal público con su folio y nombre completo para consultar sus datos y ejercer derechos ARCO (Rectificación, Cancelación, Oposición).
- Un **Administrador** ejecuta una cancelación ARCO que elimina físicamente el expediente tras la firma de un Coordinador y la firma propia del Administrador.
- Cualquier usuario autorizado consulta la bitácora de auditoría para verificar la cadena de hashes y confirmar la integridad del historial de acciones.

2. Arquitectura del Sistema

2.1 Tipo de Arquitectura

La plataforma es una **aplicación web monolítica** construida sobre Django 4.2 (Python) con el patrón *Model-View-Template* (MVT). No expone una API REST/JSON: todos los endpoints devuelven HTML renderizado del lado del servidor o redirecciones HTTP. Los formularios usan POST con CSRF token de Django. La base de datos es MySQL 8.0 y no existen dependencias de servicios externos ni microservicios.

2.2 Componentes Principales

El sistema se compone de cinco aplicaciones Django y un módulo de configuración central:

Componente	Responsabilidad
<code>config/</code>	Configuración global del proyecto Django (settings, URLs raíz, WSGI/ASGI).
<code>usuarios/</code>	Autenticación, modelo de usuario extendido con campos cifrados, gestión de roles, distribución de llaves RSA por rol, certificados X.509 de firma.
<code>expediente/</code>	Registro, lectura, edición y eliminación de expedientes cifrados con AES-256-EAX; gestión del flujo ARCO desde el lado interno (staff).
<code>auditoria/</code>	Bitácora inmutable de eventos con cadena de hashes SHA-256; solo escritura desde otras apps.
<code>cripto/</code>	Librería interna de primitivas criptográficas (RSA, AES, Scrypt, SHA-256, X.509); no expone URLs ni modelos propios; contiene el middleware de validación de certificados.
<code>portal_migrante/</code>	Portal público sin autenticación Django; acceso mediante folio + nombre completo (Scrypt); permite al migrante ver sus datos y enviar solicitudes ARCO.

Tabla 1: Componentes principales del sistema Django.

2.3 Diagrama General de Arquitectura

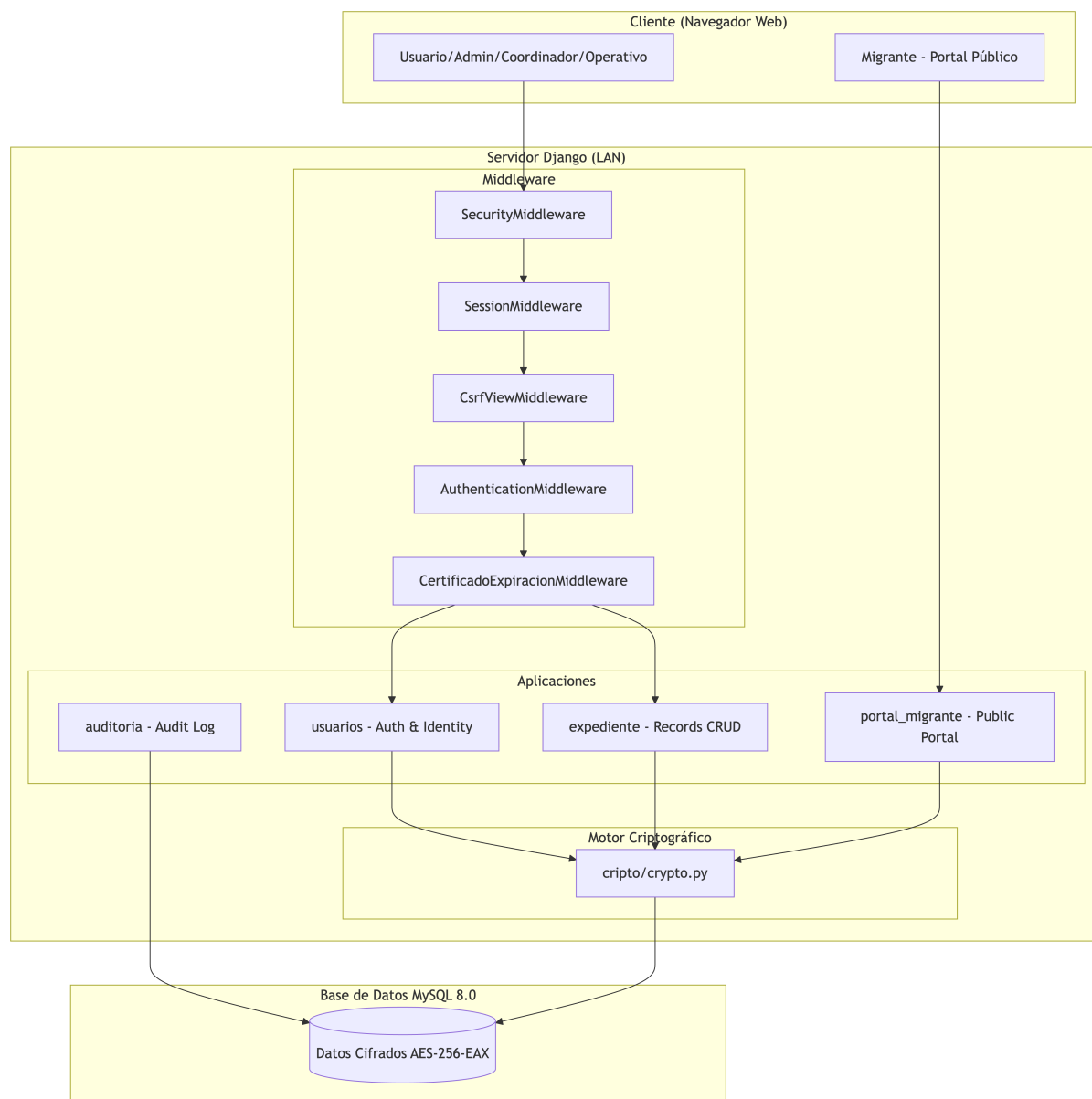


Figura 1: Diagrama general de arquitectura (monolito Django + MySQL 8.0 en LAN).

2.4 Arquitectura de Seguridad de Doble Llave

El diseño central es la **Arquitectura de Seguridad de Doble Llave**, que opera en dos capas independientes:

Capa 1 — Llave de Login (transparente): al iniciar sesión con contraseña, el sistema deriva automáticamente una clave AES-256 mediante Script y descifra la llave privada RSA-2048 del usuario, depositándola en la sesión de Django bajo la clave `_llave_privada_cache`. Simultáneamente, descifra las llaves privadas de los roles asignados al usuario y las deposita en `_llaves_rol_cache`. Este proceso es invisible para el usuario pero habilita el descifrado de expedientes durante toda la sesión activa.

Capa 2 — Llave de Firma (explícita): para operaciones críticas (editar, eliminar o verificar expedientes, y toda acción en el Panel de Administración), el sistema requiere que el usuario suba su archivo `.key` e ingrese su passphrase de 64 caracteres hexadecimales. La llave privada de firma se mantiene en sesión por 15 minutos. Pasado ese tiempo, cualquier operación crítica solicita nuevamente la llave, siguiendo el modelo de ventana temporal que implementa el SAT en su sistema de e.firma.

3. Stack Tecnológico

3.1 Lenguajes

Lenguaje	Versión	Uso
Python	3.9+	Backend completo (lógica de negocio, criptografía, ORM).
HTML5	—	Templates del frontend (Django Templates / Jinja2).
CSS3	—	Estilos con Bootstrap 5.
JavaScript	ES6	Interacciones del lado del cliente (validaciones, modales).

Tabla 2: Lenguajes de programación utilizados.

3.2 Frameworks

Framework	Versión	Propósito
Django	$\geq 4.2.0$	Framework web MVT (Model-View-Template), ORM, autenticación, permisos, sesiones.
Bootstrap	5.x	Framework CSS responsivo para el frontend.

Tabla 3: Frameworks principales del proyecto.

3.3 Bibliotecas

Paquete	Versión mínima	Propósito
<code>django-environ</code>	0.10.0	Lectura de variables de entorno desde archivo <code>.env</code> .
<code>pycryptodome</code>	3.18.0	Primitivas criptográficas: AES-EAX, RSA (PKCS1-OAEP, pkcs1_15), SHA-256.
<code>pymysql</code>	1.1.0	Driver MySQL puro-Python compatible con el ORM de Django.
<code>cryptography</code>	41.0.0	Generación y lectura de certificados X.509, serialización de llaves PEM/DER.

Tabla 4: Bibliotecas Python del proyecto.

La dependencia `pycryptodome` provee los módulos `Crypto.*` y es un fork compatible de `PyCrypto` que **no puede coexistir** con el paquete original `pycrypto`. La biblioteca `cryptography` (PyCA) se usa exclusivamente para la generación y parseo de certificados X.509 por su API de alto nivel.

4. Estructura del Proyecto

4.1 Organización de Carpetas

```

Equipo5Criptografia/
|
+-- config/                                # Nucleo del proyecto Django
|   +-- settings.py                       # Configuracion global
|   +-- urls.py                           # Router raiz
|   +-- wsgi.py                           # Punto de entrada WSGI (Gunicorn/Apache)
\-- asgi.py                                # Punto de entrada ASGI (no usado en prod)
|
+-- usuarios/                              # App: Autenticacion y gestion de usuarios
|   +-- models.py                         # Usuario, SolicitudRol, LlaveRol, AccesoLlaveRol
|   +-- views.py                          # Login, logout, panel admin, identidades
|   +-- decorators.py                     # @rol_requerido, @firma_requerida
|   +-- roles.py                          # Definicion de roles y permisos por defecto
|   +-- permissions.py                   # Listado de todos los permisos del sistema
|   +-- urls.py                           # Rutas /usuarios/*
|   \-- management/commands/
|       +-- crear_admins_base.py          # Crea admin_prod y admin_contingencia
|       \-- regenerar_admin.py            # Regenera identidad criptografica
|
+-- expediente/                            # App: Gestion de expedientes de migrantes
|   +-- models.py                         # Expediente, AccesoExpediente, EncryptedTextField
|   +-- views.py                          # CRUD de expedientes, flujo ARCO interno
|   +-- forms.py                          # Formularios de registro y edicion
|   +-- paises.py                         # Catalogo de paises (choices)
|   +-- urls.py                           # Rutas /expediente/*
|   \-- management/commands/
|       \-- inicializar_llaves_rol.py     # Genera pares RSA-2048 para los 8 roles
|
+-- auditoria/                             # App: Bitacora de eventos con cadena de hashes
|   +-- models.py                         # BitacoraEvento (auto-hash SHA-256 en cadena)
|   \-- views.py                          # Sin vistas propias; datos consultados desde admin
|
+-- cripto/                                # App: Motor criptografico central
|   +-- crypto.py                         # TODAS las primitivas criptograficas
|   +-- middleware.py                     # CertificadoExpiracionMiddleware
|   \-- models.py                         # Sin modelos propios
|
+-- portal_migrante/                       # App: Portal publico para migrantes
|   +-- models.py                         # SolicitudARCO, EncryptedTextFieldArco
|   +-- views.py                          # Acceso por folio+nombre, dashboard, ARCO
|   +-- decorators.py                     # Rate-limiting y sesion_migrante_requerida
|   \-- urls.py                           # Rutas /mi-expediente/*
|
+-- templates/                             # Templates globales
|   \-- cm_base.html                     # Template base (navbar, mensajes, estilos)
|
+-- static/                                # Archivos estaticos (logo, CSS, JS custom)
+-- test_cases/                            # Suite de 130 casos de prueba documentados
+-- certs_iniciales/                       # [ELIMINAR TRAS SETUP] .key/.cer de admins
+-- manage.py                              # CLI de Django
+-- requirements.txt                       # Dependencias Python
\-- .env                                   # Variables de entorno (NO commitear)

```

Listing 1: Árbol de directorios del proyecto.

4.2 Descripción de Módulos

config/ — Contiene la configuración global del proyecto Django. El archivo `settings.py` define las apps instaladas, el middleware, la conexión a la base de datos MySQL, las redirecciones de autenticación y el modelo de usuario personalizado (`AUTH_USER_MODEL = 'usuarios.Usuario'`). El `SECRET_KEY` leído desde `.env` es la clave raíz del cifrado transparente de base de datos.

usuarios/ — Gestiona todo el ciclo de vida de las identidades criptográficas. Incluye el modelo `Usuario` (que extiende `AbstractUser` con campos `RSA`, certificado `X.509` y campos cifrados transparentemente), el modelo `LlaveRol` (una fila por cada uno de los 8 roles con su par `RSA`), el modelo `AccesoLlaveRol` (tabla de unión que entrega la llave privada de cada rol a cada usuario autorizado), y `SolicitudRol` (gestión del flujo de solicitudes de cambio de rol). Los decoradores `@rol_requerido` y `@firma_requerida` controlan el acceso a vistas.

expediente/ — Administra los expedientes cifrados. El modelo `Expediente` almacena los datos cifrados con AES-256-EAX (campos `datos_cifrados`, `nonce`, `tag`), la firma digital, el hash del expediente y el `folio_hash` para búsqueda segura. El modelo `AccesoExpediente` distribuye la llave AES cifrada con RSA-OAEP por cada rol y creador con acceso. Las vistas implementan el CRUD cifrado y el flujo ARCO desde el lado staff.

auditoria/ — Implementa la bitácora inmutable. El modelo `BitacoraEvento` sobrescribe `save()` para calcular automáticamente el hash SHA-256 encadenado con el hash del evento anterior. El primer evento usa el literal "GENESIS" como hash previo.

cripto/ — Motor criptográfico puro (sin modelos, sin URLs). El archivo `crypto.py` encapsula todas las operaciones criptográficas en funciones puras organizadas en seis grupos: generación RSA-2048, generación X.509, derivación Script, cifrado/descifrado AES-256-EAX, hashing SHA-256 y firma digital, y cifrado transparente de BD. El middleware `CertificadoExpiracionMiddleware` fuerza logout si el certificado del usuario está expirado.

portal_migrante/ — Portal público accesible sin autenticación Django. El acceso se basa en la derivación Script del par (`folio`, `nombre_completo`). El modelo `SolicitudARCO` gestiona las solicitudes de derechos ARCO con campos cifrados transparentemente y firmas digitales de Coordinadores y Administradores.

5. Configuración del Entorno

5.1 Requisitos de Software

Software	Versión requerida
Python	3.9 o superior
MySQL Server	8.0
pip	Última versión estable
Git	2.x

Tabla 5: Requisitos de software para el entorno de desarrollo.

5.2 Variables de Entorno

El proyecto usa `django-environ` para cargar variables desde el archivo `.env` ubicado en la raíz del proyecto. Este archivo **nunca debe commitarse** al repositorio.

Variable	Tipo	Ejemplo	Descripción
SECRET_KEY	str	django-insecure-...	Clave maestra de Django. También es la clave raíz del cifrado transparente de BD. Cambiarla después de crear registros hace que todos los campos cifrados sean ilegibles.
DEBUG	bool	True	Activar solo en desarrollo. En producción debe ser False.
DB_NAME	str	casa_monarca	Nombre de la base de datos MySQL.
DB_USER	str	root	Usuario de MySQL.
DB_PASSWORD	str	mi_password	Contraseña de MySQL.
DB_HOST	str	127.0.0.1	Host del servidor MySQL.
DB_PORT	str	3306	Puerto de MySQL.

Tabla 6: Variables de entorno del archivo `.env`.

5.3 Archivo `.env` de Ejemplo

```
SECRET_KEY=django-insecure-desarrollo-local-abc123!XYZ
DEBUG=True
DB_NAME=casa_monarca
DB_USER=root
DB_PASSWORD=mi_password_local
DB_HOST=127.0.0.1
DB_PORT=3306
```

Listing 2: Archivo `.env` de ejemplo para entorno local.

5.4 Archivos de Configuración Relevantes

`config/settings.py` — Configuración principal de Django. Puntos clave:

```
1 AUTH_USER_MODEL = 'usuarios.Usuario'      # Modelo de usuario
   personalizado
2 TIME_ZONE = 'UTC'                          # Critico para timestamps de
   auditoria
3 LOGIN_URL = '/usuarios/login/'
4 LOGIN_REDIRECT_URL = '/expediente/'
5 LOGOUT_REDIRECT_URL = '/usuarios/login/'
6 DATABASES['default']['OPTIONS'] = {'charset': 'utf8mb4'} # Soporte
   espanol completo
```

Listing 3: Fragmento de `config/settings.py` con parámetros críticos.

`requirements.txt` — Dependencias Python del proyecto:

```
Django>=4.2.0
django-environ>=0.10.0
pycryptodome>=3.18.0
pymysql>=1.1.0
cryptography>=41.0.0
```

Listing 4: Archivo `requirements.txt`.

6. Instalación y Ejecución

6.1 Pasos para Levantar el Sistema

```
1 # 1. Clonar el repositorio
2 git clone <url-del-repositorio>
3 cd Equipo5Criptografia
4
5 # 2. Crear y activar entorno virtual
6 python -m venv venv
7 source venv/bin/activate          # Linux/Mac
8 # venv\Scripts\activate          # Windows
9
10 # 3. Instalar dependencias
11 pip install -r requirements.txt
12
13 # 4. Crear la base de datos MySQL
14 mysql -u root -p -e "CREATE DATABASE casa_monarca CHARACTER SET
15     utf8mb4 COLLATE utf8mb4_unicode_ci;"
16
17 # 5. Configurar el archivo .env
18 cp .env.example .env
19 # Editar .env con las credenciales de la base de datos local
20
21 # 6. Ejecutar migraciones
22 python manage.py makemigrations
23 python manage.py migrate
24
25 # 7. Inicializar llaves de rol (genera pares RSA-2048 para los 8
26     roles)
27 python manage.py inicializar_llaves_rol
28
29 # 8. Crear cuentas de administrador iniciales
30 python manage.py crear_admins_base
31
32 # 9. Levantar el servidor de desarrollo
33 python manage.py runserver
```

Listing 5: Secuencia completa de instalación y primer arranque.

6.2 Comandos Principales

Comando	Propósito
<code>python manage.py runserver</code>	Inicia el servidor de desarrollo en <code>localhost:8000</code> .
<code>python manage.py runserver 0.0.0.0:8000</code>	Inicia el servidor accesible en toda la LAN.
<code>python manage.py makemigrations</code>	Genera archivos de migración desde los modelos.
<code>python manage.py migrate</code>	Aplica las migraciones a la base de datos.
<code>python manage.py inicializar_llaves_rol</code>	Genera pares RSA-2048 para todos los roles del sistema.
<code>python manage.py crear_admins_base</code>	Crea <code>admin_prod</code> y <code>admin_contingencia</code> con acceso a todas las llaves de rol.
<code>python manage.py regenerar_admin <username></code>	Regenera la identidad criptográfica completa de un administrador.
<code>python manage.py test</code>	Ejecuta la suite completa de tests.
<code>python manage.py test usuarios</code>	Ejecuta tests de una app específica.
<code>python manage.py collectstatic</code>	Recopila archivos estáticos para producción.

Tabla 7: Comandos de gestión del proyecto Django.

6.3 Credenciales Iniciales

Tras ejecutar `crear_admins_base`, se crean dos cuentas de administrador:

Usuario	Contraseña	Propósito
<code>admin_prod</code>	<code>CasaMonarca2026!</code>	Administrador principal de producción.
<code>admin_contingencia</code>	<code>CasaMonarca2026!</code>	Administrador de respaldo/contingencia.

Tabla 8: Credenciales iniciales generadas por `crear_admins_base`.

El comando genera una carpeta `certs_iniciales/` con los archivos `.key` y `.cer` de ambos administradores. Esta carpeta debe eliminarse del servidor una vez que los administradores descarguen su identidad ZIP desde la plataforma.

7. Base de Datos

7.1 Modelo de Datos

La base de datos MySQL contiene ocho tablas principales organizadas en tres capas funcionales: identidades (usuarios y llaves), datos operativos (expedientes y accesos), y auditoría (bitácora de eventos).

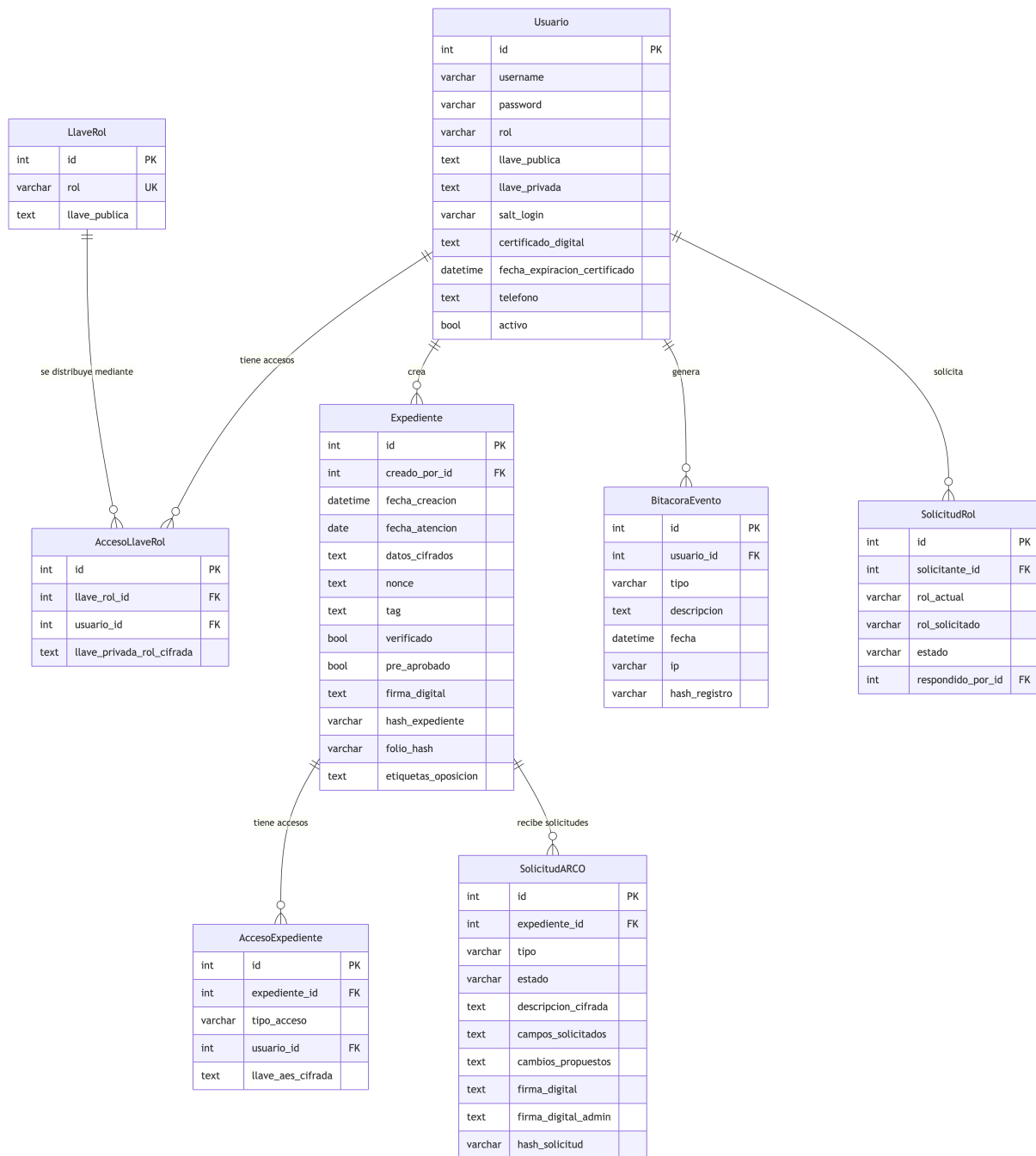


Figura 2: Diagrama entidad-relación del modelo de datos.

7.2 Tablas Principales

usuarios_usuario — Modelo de usuario extendido de Django `AbstractUser`. Los campos `llave_privada` y `telefono` se cifran transparentemente con `EncryptedTextField` y `EncryptedCharField` (AES-EAX derivado de `SECRET_KEY`). La llave privada RSA sufre doble cifrado: la capa de base de datos envuelve una segunda capa protegida con `Script(password, salt_login) + AES-EAX`.

usuarios_llaverol — Una fila por cada rol del sistema. Almacena el par RSA-2048 del rol (la llave pública en claro). La llave privada del rol no se almacena directamente aquí sino que se distribuye cifrada individualmente a cada usuario autorizado mediante `AccesoLlaveRol`.

usuarios_accesollaverol — Tabla de unión que entrega la llave privada de cada rol a cada usuario autorizado. El campo `llave_privada_rol_cifrada` contiene un paquete JSON híbrido: `{nonce, tag, llave_aes_cifrada, datos_cifrados}`, donde la llave AES se cifra con la llave pública RSA del usuario y los datos (la llave privada del rol) se cifran con AES-EAX.

expediente_expediente — Expedientes de personas atendidas. Ningún dato personal es legible en reposo. Los campos `datos_cifrados`, `nonce` y `tag` forman el paquete AES-256-EAX. El campo `folio_hash` es SHA-256(folio) para búsqueda segura desde el portal del migrante sin almacenar el folio en claro.

expediente_accesoexpediente — Control de acceso granular: una fila por cada entidad (usuario creador o rol) que puede descifrar un expediente. El campo `llave_aes_cifrada` contiene la llave AES-256 del expediente cifrada con RSA-2048 (PKCS1-OAEP) usando la llave pública del destinatario.

auditoria_bitacoraevento — Registro de auditoría con integridad garantizada por encadenamiento SHA-256. Cada evento almacena: `hash_registro = SHA-256(hash_anterior | usuario_id | tipo | descripción)`. El primer evento usa "GENESIS" como hash previo.

portal_migrante_solicitudarco — Solicitudes ARCO con campos cifrados transparentemente (`descripcion_cifrada`, `campos_solicitados`, `cambios_propuestos`, `etiqueta_oposicion`) y firmas digitales RSA del Coordinador y del Administrador.

7.3 Relaciones

Las relaciones entre tablas siguen el principio de mínimo acoplamiento criptográfico: **Usuario** se relaciona con **AccesoLlaveRol** (uno a muchos) para recibir las llaves privadas de sus roles; **Expediente** se relaciona con **AccesoExpediente** (uno a muchos) para distribuir la llave AES a cada entidad con acceso; **SolicitudARCO** referencia al **Expediente** con `SET_NULL` para conservar el registro histórico tras una cancelación. Todas las claves foráneas hacia **Usuario** usan `SET_NULL` o `PROTECT` para preservar la integridad de la auditoría.

8. API / Interfaces

El proyecto es una aplicación web Django tradicional con *server-side rendering*. **No expone una API REST/JSON.** Todos los endpoints devuelven HTML renderizado o redirecciones HTTP. Los formularios usan POST con CSRF token de Django.

8.1 Módulo usuarios — Prefijo: /usuarios/

Ruta	Métodos	Descripción	Control de acceso
/usuarios/login/	GET, POST	Formulario de login. En POST: autentica, descifra llaves de rol y las almacena en sesión.	Público
/usuarios/logout/	GET	Cierra sesión y registra evento en bitácora.	Autenticado
/usuarios/admin-panel/	GET	Panel de gestión: usuarios, solicitudes pendientes, últimos 20 eventos.	@rol_requerido + @firma_requerida
/usuarios/cambiar-rol/<pk>/	POST	Cambia rol de usuario y distribuye llaves RSA de rol cifradas.	@rol_requerido('Administrador')
/usuarios/toggle-permiso/	POST	Activa/desactiva un permiso individual de usuario.	@rol_requerido('Administrador')
/usuarios/toggle-activo/<pk>/	POST	Activa o desactiva la cuenta de un usuario.	@rol_requerido('Administrador')
/usuarios/solicitar-rol/	GET, POST	El usuario solicita un cambio de rol con justificación.	@login_required
/usuarios/responder-solicitud/<pk>/	POST	Admin aprueba o rechaza solicitud de rol.	@rol_requerido('Administrador')
/usuarios/crear-usuario/	POST	Crea usuario con identidad criptográfica completa.	@rol_requerido('Administrador')
/usuarios/regenerar-identidad/<pk>/	POST	Regenera el certificado X.509 de firma de un usuario.	@rol_requerido('Administrador')
/usuarios/revocar-certificado/<pk>/	POST	Elimina el certificado de firma.	@rol_requerido('Administrador')
/usuarios/reset-password-admin/<pk>/	POST	Resetea contraseña, renueva par RSA y redistribuye llaves de rol.	@rol_requerido('Administrador')
/usuarios/descargar-certificado/	GET	Descarga ZIP con .cer + .key (una sola vez por sesión).	@rol_requerido('Administrador')
/usuarios/ingresar-firma/	GET, POST	Sube archivo .key + passphrase para desbloquear firma por 15 min.	@login_required
/usuarios/cambiar-password/	GET, POST	Cambia contraseña y recifra la llave de login.	@login_required

Tabla 9: Endpoints del módulo usuarios.

Parámetros POST principales:

```

POST /usuarios/crear-usuario/
  username=string
  password=string (min 8 caracteres)
  first_name=string
  last_name=string
  rol=string (Administrador|Coordinador_*|Operativo|Usuario)

POST /usuarios/cambiar-rol/<pk>/
  nuevo_rol=string

```

```

POST /usuarios/toggle-permiso/
  user_id=int
  codename=string

POST /usuarios/responder-solicitud/<pk>/
  accion=aprobar|rechazar
  respuesta=string (opcional)

```

Listing 6: Parámetros POST del módulo usuarios.

8.2 Módulo expediente — Prefijo: /expediente/

Ruta	Métodos	Descripción	Control
/expediente/dashboard/	GET	Panel principal con estadísticas y accesos rápidos.	@login_required
/expediente/registrar/	GET, POST	Registro de migrante. Cifra datos con AES-256-EAX.	Roles con permiso de registro.
/expediente/expedientes/	GET	Lista de expedientes descifrados según llaves de rol en sesión.	Roles con lectura.
/expediente/expedientes/verificar/	GET, POST	Coordinadores verifican y firman expedientes.	@firma_requerida
/expediente/expedientes/pre-aprobar/<pk>/	POST	Operativo pre-aprueba expediente.	Rol Operativo
/expediente/expedientes/editar/<pk>/	GET, POST	Descifra, edita y re-cifra el expediente.	@firma_requerida
/expediente/expedientes/eliminar/<pk>/	POST	Elimina expediente y registra en bitácora.	@firma_requerida
/expediente/arco/	GET	Lista solicitudes ARCO activas (vista interna).	Roles con acceso ARCO.
/expediente/arco/responder/<pk>/	GET, POST	Operativo aprueba/rechaza solicitud ARCO.	Rol Operativo
/expediente/arco/firmar/<pk>/	GET, POST	Coordinador firma solicitud ARCO aprobada.	@firma_requerida
/expediente/arco/ejecutar/<pk>/	POST	Admin ejecuta cancelación (elimina expediente).	@rol_requerido + @firma_requerida

Tabla 10: Endpoints del módulo expediente.

8.3 Portal Migrante — Prefijo: /mi-expediente/

Este módulo opera **sin autenticación Django**. El acceso se basa en folio + nombre_completo con rate-limiting.

Ruta	Métodos	Descripción
/mi-expediente/	GET, POST	Formulario de acceso por folio y nombre. Rate-limited: 5 intentos / 15 min / IP.
/mi-expediente/mis-datos/	GET	Muestra los datos descifrados del expediente del migrante.
/mi-expediente/privacidad/	GET	Muestra el aviso de privacidad de Casa Monarca.
/mi-expediente/arco/	GET, POST	Formulario para crear solicitud ARCO.
/mi-expediente/solicitudes/	GET	Historial de solicitudes ARCO del migrante.
/mi-expediente/salir/	GET	Cierra la sesión del portal migrante.

Tabla 11: Endpoints del portal público del migrante.

Parámetros POST del portal:

```

POST /mi-expediente/
  folio=CM-YYYYMMDD-XXXX
  nombre_completo=string

POST /mi-expediente/arco/
  tipo=rectificacion|cancelacion|oposicion
  descripcion=string
  campos_solicitados=string (JSON, solo para rectificacion)

```

Listing 7: Parámetros POST del portal migrante.

8.4 Router Raíz — config/urls.py

```

1 urlpatterns = [
2     path('admin/', admin.site.urls),
3     path('usuarios/', include('usuarios.urls')),
4     path('expediente/', include('expediente.urls')),
5     path('mi-expediente/', include('portal_migrante.urls')),
6 ]

```

Listing 8: Configuración del router raíz de URLs.

No existe una vista en / (raíz). El punto de entrada funcional es /usuarios/login/.

9. Flujos y Lógica de Negocio**9.1 Flujo de Login y Desbloqueo Criptográfico**

El login no solo valida la contraseña con Django (PBKDF2): además deriva la clave Script y descifra la llave privada RSA del usuario para depositarla en la sesión.

USUARIO	SERVIDOR	BASE DE DATOS
-- username + password ----->		
	-- SELECT usuario WHERE username -->	
	<-- {rol, salt_login, llave_privada,	
	certificado_digital, ...} -----	

```

|         +- [1] Django authenticate()
|         |     verifica PBKDF2(password) vs
|         |     campo password
|         |
|         +- [2] Descifrar llave_privada:
|         |     a) EncryptedTextField descifra
|         |         AES-EAX(SECRET_KEY) -> PEM
|         |         cifrado con Scrypt
|         |     b) clave = Scrypt(password,
|         |         salt_login)
|         |     c) AES-EAX(clave) -> PEM puro
|         |         -> sesion: _llave_privada_cache
|         |
|         +- [3] Cargar llaves de rol:
|         |     Para cada AccesoLlaveRol:
|         |     a) JSON.parse(paquete_hibrido)
|         |     b) RSA-OAEP(llave_privada_cache)
|         |         -> llave_aes_rol
|         |     c) AES-EAX(llave_aes_rol)
|         |         -> llave_privada_rol_PEM
|         |         -> sesion: _llaves_rol_cache[rol]
|         |
|         +- [4] Registrar BitacoraEvento:
|         |     tipo=LOGIN, hash=SHA256(prev|..)
|         |
| <-- redirect /expediente/ -----|

```

Listing 9: Flujo de login y desbloqueo criptográfico.

Datos almacenados en sesión tras login exitoso:

Clave de sesión	Expiración	Contenido
_llave_privada_cache	Logout	Llave privada RSA-2048 PEM del usuario.
_llaves_rol_cache	Logout	Dict {rol: llave_privada_PEM}.
llave_privada_firma	Logout o 15 min	Llave privada del archivo .key (SAT).
tiempo_firma_reciente	Logout o expiración	Timestamp UNIX de cuando se subió el .key.

Tabla 12: Variables almacenadas en la sesión de Django tras un login exitoso.

9.2 Flujo de Creación de Expediente

```
USUARIO (cualquier rol con permiso de registro)
|
+- [1] Llena formulario con datos del migrante
|
+- [2] Backend genera llave_aes = os.urandom(32) [256 bits]
|
+- [3] Cifrar datos:
|     cipher = AES.new(llave_aes, AES.MODE_EAX)
|     datos_cifrados, tag = cipher.encrypt_and_digest(JSON(formulario))
|
+- [4] Calcular hash:
|     hash_expediente = SHA-256(datos_cifrados)
|
+- [5] Generar folio unico: CM-YYYYMMDD-XXXX
|     folio_hash = SHA-256(folio)
|
+- [6] Distribuir acceso (una entrada por entidad):
```

```

|      Para el CREADOR:
|      llave_aes_cifrada = RSA-OAEP(llave_publica_usuario, llave_aes)
|      AccesoExpediente(tipo='Creador', usuario=creador)
|      Para cada ROL (7 roles):
|      llave_aes_cifrada = RSA-OAEP(llave_publica_rol, llave_aes)
|      AccesoExpediente(tipo=nombre_rol, usuario=None)
|      Para el MIGRANTE:
|      clave_folio = Scrypt(folio:nombre, salt=SHA-256(folio))
|      llave_aes_migrante = AES-EAX(clave_folio).encrypt(llave_aes)
|      AccesoExpediente(tipo='Migrante', usuario=None)
|
+- [7] Guardar Expediente en BD
|
\-- [8] Registrar BitacoraEvento: tipo=CREACION

```

Listing 10: Flujo de creación y cifrado de un expediente.

9.3 Flujo de Operación Crítica (Firma Digital)

Requerida por el decorador `@firma_requerida` para editar expediente, eliminar expediente, verificar expediente y gestión en el Panel de Administración.

```

COORDINADOR (con .key cargado en sesion, valido 15 min)
|
+- [1] @firma_requerida verifica timestamp en sesion
|      Si (time.time() - tiempo_firma_reciente) > 900:
|      redirect -> /usuarios/ingresar-firma/
|
+- [2] Calcular hash del expediente:
|      hash_exp = SHA-256(datos_cifrados)
|
+- [3] Firmar con llave privada de firma:
|      firma = RSA-2048-PKCS1v15(llave_privada_firma, SHA-256(hash_exp))
|
+- [4] Guardar firma en BD:
|      Expediente.firma_digital = base64(firma)
|      Expediente.verificado = True
|
\-- [5] Registrar BitacoraEvento: tipo=verificacion_expediente

```

Listing 11: Flujo de operación crítica con firma digital.

9.4 Flujo de Acceso del Migrante por Folio

MIGRANTE	SERVIDOR
-- POST folio + nombre ----->	
	+- [1] Rate-limit: 5 intentos/15min por IP
	+- [2] folio_hash = SHA-256(folio)
	Expediente.objects.filter(folio_hash=...)
	+- [3] AccesoExpediente WHERE tipo='Migrante'
	-> llave_aes_cifrada
	+- [4] clave_folio = Scrypt(folio:nombre,
	salt=SHA-256(folio))
	llave_aes = AES-EAX(clave_folio)
	.decrypt(llave_aes_cifrada)
	Si falla -> "Folio o nombre incorrecto"
	+- [5] datos = AES-EAX(llave_aes, nonce, tag)
	.decrypt(datos_cifrados)

```
|
|                                     +- [6] Sesion (15 min sliding window):
|                                     |     _migrante_expediente_id, _migrante_datos
|                                     |
|<-- redirect /mis-datos/ -----|
```

Listing 12: Flujo de autenticación del migrante mediante folio y nombre.

9.5 Flujo ARCO — Rectificación

```
[1] MIGRANTE -> POST tipo='rectificacion', descripcion, campos_solicitados
    -> solicitud.estado = 'pendiente'

[2] OPERATIVO -> GET /expediente/arco/responder/<pk>/
    Descifra expediente con llaves de rol en sesion.
    Muestra formulario prellenado + solicitud del migrante.
    POST aprobar:
        diff = {campo: valor_nuevo para campos modificados}
        solicitud.cambios_propuestos = AES-EAX(SECRET_KEY).encrypt(JSON(diff))
        solicitud.estado = 'aprobada_operativo'

[3] COORDINADOR (con .key cargado) -> GET /expediente/arco/firmar/<pk>/
    POST firmar:
        [a] firma = RSA-PKCS1v15(llave_privada_firma,
                                "ARCO-{pk}-rectificacion-{hash}-{ts}")
        [b] Descifra expediente con llave de rol
        [c] Aplica diff: datos.update(cambios_propuestos)
        [d] Re-cifra con misma llave AES: nuevo paquete AES-EAX
        [e] Actualiza hash_expediente, firma_digital, verificado=True
        [f] solicitud.estado = 'ejecutada'
```

Listing 13: Flujo completo de una solicitud ARCO de Rectificación.

9.6 Flujo ARCO — Cancelación (5 estados, 4 actores)

```
[1] MIGRANTE:      POST tipo='cancelacion' -> estado='pendiente'
[2] OPERATIVO:    pre-aprueba -> estado='aprobada_operativo'
[3] COORDINADOR:  firma con .key -> estado='firmada_coordinador'
[4] ADMINISTRADOR: firma con .key + ejecuta:
    - Firma admin = RSA-PKCS1v15(
        "ARCO-CANCEL-{pk}-{exp_id}-{hash}-{ts}")
    - BitacoraEvento(tipo='expediente_cancelado_arco')
    - SolicitudARCO.objects.filter(expediente=exp)
      .update(expediente=None)
    - expediente.delete()
    -> estado='ejecutada'
[5] EXPEDIENTE BORRADO FISICAMENTE
    (Solicitudes ARCO conservan registro historico con expediente=NULL)
```

Listing 14: Flujo de cancelación ARCO con firma dual (Coordinador + Administrador).

9.7 Cadena de Auditoría

Cada evento en `BitacoraEvento` es un eslabón de una cadena hash que detecta manipulación retroactiva:

$$\text{hash_registro}[N] = \text{SHA-256}(\text{hash_registro}[N-1] \parallel \text{usuario_id} \parallel \text{tipo} \parallel \text{descripcion} \parallel \text{fecha}) \quad (1)$$

Para verificar integridad: recalculamos $\text{hash_registro}[i]$ para $i = 0 \dots N$. Si alguno no coincide con el almacenado, el registro fue manipulado.

10. Ejemplos de Uso

10.1 Cifrar y Descifrar Datos de un Expediente

```

1 from cripto.cripto import (
2     cifrar_datos_sin_rsa,
3     descifrar_datos_con_aes_existente,
4     cifrar_llave_aes,
5     descifrar_llave_aes
6 )
7 from usuarios.models import LlaveRol
8
9 # --- CIFRADO (al registrar un expediente) ---
10 datos_migrante = {
11     'nombre': 'Ana Garcia Lopez',
12     'pais_origen': 'Honduras',
13     'fecha_nacimiento': '1990-05-15',
14     'situacion': 'Solicitante de asilo',
15 }
16
17 # cifrar_datos_sin_rsa genera llave AES aleatoria de 256 bits
18 paquete = cifrar_datos_sin_rsa(datos_migrante)
19 # paquete = {
20 #     'datos_cifrados': '<base64>',
21 #     'nonce': '<base64>',
22 #     'tag': '<base64>',
23 #     'llave_aes': b'\x...' (32 bytes en RAM)
24 # }
25
26 # Cifrar la llave AES con la llave publica del rol
27 llave_rol = LlaveRol.objects.get(rol='Coordinador_Legal')
28 llave_aes_cifrada = cifrar_llave_aes(paquete['llave_aes'],
29                                     llave_rol.llave_publica)
30
31 # --- DESCIFRADO (al leer un expediente) ---
32 # La llave privada del rol esta en la sesion del usuario autenticado
33 llave_privada_rol = request.session['_llaves_rol_cache']['Coordinador_Legal']
34
35 llave_aes = descifrar_llave_aes(llave_aes_cifrada, llave_privada_rol)
36 datos = descifrar_datos_con_aes_existente(
37     paquete['datos_cifrados'],
38     paquete['nonce'],
39     paquete['tag'],
40     llave_aes
41 )

```

```
42 # datos == {'nombre': 'Ana Garcia Lopez', ...}
```

Listing 15: Ejemplo de cifrado y descifrado de un expediente completo.

10.2 Firma Digital y Verificación

```
1 from cripto.cripto import firmar, verificar_firma
2
3 # La llave privada de firma se desbloquea en la vista ingresar_firma
4 llave_privada_firma = request.session.get('llave_privada_firma')
5
6 # --- FIRMA ---
7 datos_a_firmar = (f"expediente:{expediente.pk}"
8                  f"|hash:{expediente.hash_expediente}")
9 firma_b64 = firmar(datos_a_firmar, llave_privada_firma)
10 expediente.firma_digital = firma_b64
11 expediente.save()
12
13 # --- VERIFICACION ---
14 from cryptography.x509 import load_pem_x509_certificate
15 from cryptography.hazmat.primitives.serialization import (
16     Encoding, PublicFormat
17 )
18
19 cert = load_pem_x509_certificate(
20     coordinador.certificado_digital.encode('utf-8')
21 )
22 llave_publica_pem = cert.public_key().public_bytes(
23     Encoding.PEM, PublicFormat.SubjectPublicKeyInfo
24 ).decode('utf-8')
25
26 es_valida = verificar_firma(datos_a_firmar, firma_b64,
27                             llave_publica_pem)
28 # es_valida == True si la firma es autentica
```

Listing 16: Ejemplo de firma digital y verificación con certificado X.509.

10.3 Derivación de Clave para Portal del Migrante

```
1 from cripto.cripto import derivar_clave_folio
2
3 # El migrante proporciona su folio y nombre completo
4 folio = "CM-20260607-AB12"
5 nombre_completo = "Ana Garcia Lopez"
6
7 # Derivacion deterministica: Scrypt(folio:nombre_lower, salt=SHA256(
8   folio))
9 clave_folio = derivar_clave_folio(folio, nombre_completo)
10 # clave_folio: bytes de 32 bytes (256 bits)
11
12 # Con esta clave se descifra la llave AES almacenada en
13 # AccesoExpediente tipo='Migrante'
```

Listing 17: Derivación determinística de clave para el portal del migrante.

10.4 Cifrado Transparente de Base de Datos

```
1 from usuarios.models import Usuario
2
3 # Al leer, el campo se descifra automaticamente:
4 usuario = Usuario.objects.get(username='voluntaria_1')
5 print(usuario.telefono) # "555-123-4567" (texto plano en memoria)
6
7 # Al guardar, se cifra automaticamente antes de escribir en MySQL:
8 usuario.telefono = "555-987-6543"
9 usuario.save()
10 # En la BD queda: base64(nonce + tag + ciphertext_AES_EAX)
```

Listing 18: Comportamiento del cifrado transparente en campos ORM.

11. Manejo de Errores

11.1 Estrategia de Errores

El sistema implementa dos estrategias complementarias para el manejo de errores: mensajes de usuario a través del framework de mensajes de Django (`django.contrib.messages`) y registro de eventos de negocio a través de la bitácora de auditoría (`BitacoraEvento`).

Los errores criptográficos (llave incorrecta, certificado expirado, firma inválida) se capturan con bloques `try/except` (`ValueError`, `TypeError`) y se traducen a mensajes genéricos que no revelan detalles internos al usuario. El principio rector es que los mensajes de error **no deben filtrar información** sobre la existencia de usuarios, la validez parcial de credenciales, o el estado de las llaves criptográficas.

11.2 Mensajes Comunes

Contexto	Mensaje al Usuario	Causa
Login fallido	“Usuario o contraseña incorrectos.”	Credenciales inválidas (mismo mensaje para usuario inexistente).
Cuenta desactivada	“Tu cuenta está desactivada. Contacta al administrador.”	Campo <code>activo=False</code> en el modelo <code>Usuario</code> .
Sesión criptográfica inválida	“Tu sesión criptográfica no fue desbloqueada correctamente...”	Llave privada corrupta o salt inválido al momento del login.
Certificado expirado	“Tu sesión ha sido bloqueada porque tu identidad criptográfica expiró.”	Middleware detecta <code>fecha_expiracion_certificado < now()</code> .
Firma expirada	“Operación crítica protegida. Por favor, sube tu llave de firma.”	<code>time.time() - tiempo_firma_reciente > 900</code> .
Llave <code>.key</code> incorrecta	“Llave de firma incorrecta.”	Passphrase no descifra el archivo PKCS#8.
Acceso denegado	“No tienes permisos para acceder a esta sección.”	Rol no está en la lista de <code>@rol_requerido</code> .
Portal migrante fallido	“Folio o nombre incorrecto.”	Script no puede descifrar la llave AES.
Rate-limit portal	“Demasiados intentos. Intenta de nuevo en 15 minutos.”	5 intentos fallidos desde la misma IP en 15 min.

Tabla 13: Mensajes de error y sus causas técnicas.

11.3 Registro (Logs)

Logging técnico: el archivo `settings.py` no define una configuración `LOGGING` explícita. Django usa su configuración por defecto: en desarrollo (`DEBUG=True`) los errores se imprimen en la consola (`stdout/stderr`); en producción (`DEBUG=False`) Django envía errores 500 por correo a las direcciones en `ADMINS` y los escribe en `stderr` del proceso servidor.

Auditoría de negocio: el sistema implementa su propio registro de eventos críticos a través de `BitacoraEvento`. Este no es un sistema de logging técnico sino de trazabilidad de negocio que registra: login/logout, cambios de rol y permisos, creación/edición/eliminación de expedientes, flujo completo de solicitudes ARCO, y accesos del portal migrante (exitosos y fallidos). Cada registro incluye usuario, tipo de evento, descripción, IP del cliente, timestamp y hash SHA-256 encadenado.

12. Pruebas

12.1 Tipos de Tests

El proyecto implementa una estrategia de pruebas funcional basada en ejecución asistida por agente e impulsada por evidencia visual, documentada en la carpeta `test_cases/`. Se diseñaron y ejecutaron 151 casos de prueba organizados en ocho categorías funcionales:

Categoría	Casos	Cobertura
Autenticación	13	Login exitoso por rol (Administrador, Coordinador, Operativo, Usuario), login fallido (contraseña incorrecta, usuario inexistente, cuenta desactivada), logout, acceso sin sesión, sesión ya activa, llave privada corrupta.
Gestión de Usuarios	22	Creación con identidad criptográfica completa por rol, validaciones de formulario, descarga de ZIP con <code>.cer/.key</code> , toggle activo/inactivo, reset de contraseña con re-distribución de llaves de rol, cambio de contraseña propia.
Roles y Permisos	20	Cambio de rol con re-distribución de llaves, permisos individuales vs. heredados, flujo completo de solicitudes de cambio de rol (pendiente aprobada/rechazada), notificaciones de badge.
Certificados y Firma	22	Ingreso de archivo <code>.key</code> con passphrase, ventana de validez de 15 minutos, correspondencia con certificado X.509, firma y verificación RSA, expiración de certificado, flujo de re-ingreso de firma tras expiración.
Expedientes	25	CRUD cifrado por rol, distribución de llaves AES por <code>AccesoExpediente</code> , descifrado selectivo, filtros (estado, creador, fechas), comportamiento de columnas de acción por rol, fallback de descifrado, aislamiento de acceso.
Verificación de Expedientes	10	Flujo completo de pre-aprobación y firma digital, recálculo de hash SHA-256, verificación de firma RSA con llave pública del firmante, integridad del campo <code>hash_expediente</code> .
Auditoría	19	Registro de eventos clave en <code>BitacoraEvento</code> , cadena de hashes SHA-256, detección de manipulación del encadenamiento, limpieza de sesión criptográfica en logout, verificación de cambio de contraseña en BD.
Seguridad Criptográfica	20	Ilegibilidad de <code>datos_cifrados</code> en reposo, integridad AES-EAX, aislamiento entre roles, invariante de re-uso de llave AES en edición, restricciones del decorador <code>@firma_requerida</code> , formato del archivo <code>.key</code> (PKCS#8 DER cifrado).

Tabla 14: Categorías y cobertura de los 151 casos de prueba.

12.2 Metodología de Ejecución: Playwright MCP

Las pruebas se ejecutaron mediante el **servidor MCP de Playwright** (*Model Context Protocol*) operado por un agente Claude Code. Este mecanismo permite al agente controlar un navegador Chromium de forma programática a través de herramientas de alto nivel, sin depender de scripts monolíticos ejecutados externamente. El flujo de ejecución por cada caso de prueba fue el siguiente:

1. El agente recibe la especificación del caso (`test_cases/_specs/TC-XX-YY.md`) y configura las precondiciones de base de datos directamente mediante el ORM de Django cuando es necesario (por ejemplo, corromper una llave privada o desactivar una cuenta).
2. El agente invoca las herramientas MCP de Playwright para navegar, interactuar con la interfaz y capturar evidencia paso a paso.
3. Por cada paso definido en la especificación se captura un screenshot nombrado `TC-XX-YY_paso-N.png`; al concluir la ejecución se genera un archivo animado `TC-XX-YY.webp` con todos los frames del caso.
4. El agente redacta el documento de resultados `TC-XX-YY.md` con la tabla de pasos, resultado obtenido, verificación en BD (si aplica) y galería de evidencia.

Las herramientas MCP de Playwright empleadas durante la ejecución son:

Herramienta MCP	Uso
<code>browser_navigate</code>	Navegar a una URL y esperar carga completa.
<code>browser_snapshot</code>	Capturar el árbol de accesibilidad de la página en formato YAML (archivos <code>.playwright-mcp/*.yaml</code>); base de las aserciones del agente.
<code>browser_type</code>	Escribir texto en un campo de formulario identificado por selector CSS o ref.
<code>browser_click</code>	Hacer clic sobre un elemento.
<code>browser_take_screenshot</code>	Guardar imagen PNG del estado actual del navegador como evidencia de paso.
<code>browser_wait_for</code>	Esperar a que una URL o condición se satisfaga antes de continuar.
<code>browser_file_upload</code>	Cargar archivos <code>.key</code> en los formularios de ingreso de firma.

Tabla 15: Herramientas del servidor MCP de Playwright utilizadas en la ejecución de pruebas.

Los snapshots de árbol de accesibilidad quedaron persistidos en la carpeta `.playwright-mcp/` del repositorio (archivos `page-<ISO8601>.yaml`), lo que permite auditar exactamente el estado del DOM que el agente observó en cada interacción.

12.3 Estructura de la Carpeta `test_cases/`

```
test_cases/
+-- _TEMPLATE.md                # Plantilla estandar para nuevos casos
+-- _specs/                     # Especificaciones previas a la ejecucion
|   +-- TC-01-01.md             # Guion fijo de pasos y aserciones
|   +-- TC-01-02.md
|   \-- ...
+-- test_cases_INDEX.md         # Indice maestro (generado por merge_indexes.py)
+-- merge_indexes.py            # Script que fusiona _index.md de subcarpetas
+-- generate_evidence.py        # Script auxiliar: recompila evidencia .webp
+-- take_screenshot.py          # Utilidad de captura individual (prototipo)
+-- 01_autenticacion/          # 13 casos (TC-01-01 a TC-01-13)
|   +-- TC-01-01.md             # Resultado documentado
|   +-- TC-01-01_paso-1.png      # Evidencia por paso
|   +-- TC-01-01_paso-2.png
|   +-- TC-01-01.webp           # Animacion resumen del caso
|   +-- _index.md               # Indice de la categoria
|   \-- ...
+-- 02_gestion_usuarios/        # 22 casos
+-- 03_roles_y_permisos/        # 20 casos
+-- 04_certificados_y_firma/    # 22 casos
+-- 05_expedientes/            # 25 casos
+-- 06_verificacion_expedientes/ # 10 casos
+-- 07_auditoria/              # 19 casos
\-- 08_seguridad_criptografica/ # 20 casos
```

Listing 19: Estructura de la carpeta `test_cases/`.

El índice maestro `test_cases_INDEX.md` se genera automáticamente ejecutando `merge_indexes.py`, que recorre cada `_index.md` de subcarpeta y consolida las tablas de casos en un único documento:

```
1 python test_cases/merge_indexes.py
```

Listing 20: Regeneración del índice maestro de casos de prueba.

El script auxiliar `generate_evidence.py` permite recompilar las imágenes animadas `.webp` a partir de los PNG de pasos ya existentes, empleando la biblioteca `Pillow` y la API síncrona de `Playwright`. Añade además un marco de navegador estilizado (barra de título macOS, botones de semáforo, barra de URL con candado) a cada frame para mejorar la legibilidad de la evidencia:

```

1 def add_browser_frame(image_path: str, title: str, url: str) -> None:
2     """Superpone una barra de navegador estilizada sobre el PNG."""
3
4 def save_animated_webp(
5     image_paths: list[str],
6     output_path: str,
7     duration: int = 2000
8 ) -> None:
9     """Compila una lista de PNG en un WebP animado y elimina los PNG
    temporales."""

```

Listing 21: Firma de las funciones de utilidad en `generate_evidence.py`.

12.4 Ejemplo de Caso de Prueba

TC-01-01: Login exitoso con credenciales válidas de Administrador

- **Precondición:** usuario `admin_prod` creado con `crear_admins_base`, cuenta activa; servidor en `http://127.0.0.1:8000`; navegador sin sesión previa.
- **Pasos:**
 1. `browser_navigate /usuarios/login/`; capturar paso-1 (formulario vacío).
 2. `browser_type` en `#id_username = admin_prod`; `browser_type` en `#id_password = adminprod`; capturar paso-2 (formulario lleno).
 3. `browser_click` en `button.btn-login`; `browser_wait_for URL **/expediente/dashboard/`; capturar paso-3 (Dashboard con badge de rol).
 4. `browser_click` en `#toggle-status` (panel *Seguridad y Permisos*); capturar paso-4 (panel desplegado con estado criptográfico).
- **Resultado esperado:** redirect al Dashboard; la barra superior muestra el badge **Administrador**; el panel *Estado Criptográfico* indica **Llaves RSA: Activas** y **Certificado X.509: Activo**; la tarjeta *Gestionar usuarios* es visible.
- **Resultado obtenido:** URL final `/expediente/dashboard/`; badge *Administrador Produccion*; panel criptográfico con llaves RSA activas y certificado X.509 activo; permisos efectivos (Editar, Eliminar, Exportar, Visualizar expedientes, Gestionar usuarios, Ver bitácora, Registrar) presentes. **Estado: PASS.**

13. Despliegue

13.1 Opción A: Gunicorn + Nginx (VPS / Servidor Dedicado)

```

1 # 1. Instalar Gunicorn
2 pip install gunicorn

```

```

3
4 # 2. Levantar Gunicorn
5 gunicorn config.wsgi:application \
6     --bind 127.0.0.1:8000 \
7     --workers 3 \
8     --timeout 120 \
9     --access-logfile /var/log/casa_monarca/gunicorn_access.log \
10    --error-logfile /var/log/casa_monarca/gunicorn_error.log \
11    --daemon
12
13 # 3. Recopilar archivos estaticos
14 python manage.py collectstatic --noinput

```

Listing 22: Despliegue con Gunicorn en un VPS.

Configuración de Nginx como proxy inverso:

```

1 server {
2     listen 80;
3     server_name tudominio.com;
4     return 301 https://$server_name$request_uri;
5 }
6
7 server {
8     listen 443 ssl;
9     server_name tudominio.com;
10
11     ssl_certificate /etc/letsencrypt/live/tudominio.com/fullchain
12     .pem;
13     ssl_certificate_key /etc/letsencrypt/live/tudominio.com/privkey.
14     pem;
15
16     location /static/ {
17         alias /ruta/al/proyecto/staticfiles/;
18     }
19
20     location / {
21         proxy_pass http://127.0.0.1:8000;
22         proxy_set_header Host $host;
23         proxy_set_header X-Real-IP $remote_addr;
24         proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for
25     ;
26         proxy_set_header X-Forwarded-Proto $scheme;
27     }
28 }

```

Listing 23: Configuración de Nginx con HTTPS y proxy inverso a Gunicorn.

13.2 Opción B: Hosting Compartido con cPanel (HostGator / Apache)

Si el servidor soporta Phusion Passenger (cPanel moderno): configurar la aplicación Python desde el menú “Setup Python App”, apuntar el *Application startup file* a `config/wsgi.py` y el *Application entry point* a `application`.

Si solo soporta FastCGI, crear un archivo `dispatch.fcgi`:

```

1 #!/usr/bin/env python

```

```

2 import sys, os
3 sys.path.insert(0, '/home/usuario/Equipo5Criptografia')
4 os.environ['DJANGO_SETTINGS_MODULE'] = 'config.settings'
5 from django.core.handlers.wsgi import WSGIHandler
6 from flup.server.fcgi import WSGIServer
7 application = WSGIHandler()
8 if __name__ == '__main__':
9     WSGIServer(application).run()

```

Listing 24: Adaptador FastCGI para hosting compartido con flup6.

Requiere instalar flup6 (pip install flup6).

13.3 Entornos

Entorno	DEBUG	ALLOWED_HOSTS	Descripción
Desarrollo local	True	['localhost', '127.0.0.1']	Servidor de desarrollo de Django (runserver).
LAN (demo)	True	['*']	runserver 0.0.0.0:8000 accesible en red local.
Producción	False	['tudominio.com']	Gunicorn + Nginx con HTTPS.

Tabla 16: Configuración por entorno de ejecución.

13.4 Consideraciones Críticas

- El archivo `.env` debe estar **fuera** del directorio público (`public_html`).
- La `SECRET_KEY` de producción debe ser única, aleatoria y de al menos 50 caracteres. Cambiarla invalida todos los campos cifrados existentes.
- La carpeta `certs_iniciales/` debe eliminarse del servidor tras el setup inicial.
- Ejecutar `collectstatic` tras cada deploy.
- Configurar `STATIC_ROOT = BASE_DIR / 'staticfiles'` en producción.
- Las sesiones Django se almacenan en base de datos; el directorio de sesiones debe tener permisos de escritura.

14. Seguridad

14.1 Autenticación

El sistema implementa autenticación en múltiples capas:

Capa Django (PBKDF2): la contraseña del usuario se hashea con PBKDF2-SHA256 usando la implementación estándar de Django (`django.contrib.auth`). Este hash se almacena en el campo `password` del modelo `Usuario`.

Capa criptográfica (Scrypt + AES-EAX): simultáneamente, la contraseña se usa para derivar una clave AES-256 mediante Scrypt ($N = 16384, r = 8, p = 1, dklen = 32$) que descifra la llave privada RSA del usuario. Sin la contraseña correcta, la llave privada permanece inaccesible.

Capa de firma (PKCS#8 + X.509): para operaciones críticas, se requiere el archivo `.key` (llave privada exportada en PKCS#8 cifrado con AES-256-CBC + Scrypt) protegido por una passphrase de 64 caracteres hexadecimales. El middleware `CertificadoExpiracionMiddleware` fuerza logout si el certificado X.509 está vencido.

14.2 Manejo de Datos Sensibles

Todos los datos personales de migrantes se almacenan cifrados con AES-256-EAX. Las capas de cifrado son:

Capa	Qué protege	Algoritmo	Llave
1	Datos del expediente (PII)	AES-256-EAX	Llave AES aleatoria de 32 bytes por expediente.
2	Llave AES del expediente	RSA-2048-OAEP	Llave pública del rol.
3	Llave privada del rol	RSA-2048-OAEP + AES-EAX	Llave pública del usuario.
4	Llave privada del usuario	Scrypt + AES-256-EAX	Contraseña del usuario.
Paralela	Campos administrativos (teléfono, descripciones ARCO)	AES-256-EAX	SHA-256(SECRET_KEY).

Tabla 17: Capas de cifrado de datos sensibles en el sistema.

La llave AES del expediente **nunca se almacena en claro**; se genera en RAM, se usa para cifrar y se descarta. La llave privada del usuario nunca sale de la sesión en claro. El archivo `.key` es descargable una sola vez en el momento de creación del usuario.

14.3 Middleware y Cabeceras de Seguridad

```

1 MIDDLEWARE = [
2     'django.middleware.security.SecurityMiddleware',           #
3     'django.contrib.sessions.middleware.SessionMiddleware',
4     'django.middleware.common.CommonMiddleware',
5     'django.middleware.csrf.CsrfViewMiddleware',             #
6     'django.contrib.auth.middleware.AuthenticationMiddleware',
7     'django.contrib.messages.middleware.MessageMiddleware',
8     'django.middleware.clickjacking.XFrameOptionsMiddleware', # X-
9     'cripto.middleware.CertificadoExpiracionMiddleware',      #
10 ]
```

Listing 25: Configuración del stack de middleware de Django.

14.4 Configuración de Seguridad Recomendada para Producción

```

1 SECURE_SSL_REDIRECT = True
```

```

2 SECURE_HSTS_SECONDS = 31536000
3 SECURE_HSTS_INCLUDE_SUBDOMAINS = True
4 SECURE_HSTS_PRELOAD = True
5 SESSION_COOKIE_SECURE = True
6 CSRF_COOKIE_SECURE = True
7 ALLOWED_HOSTS = ['tudominio.com', 'www.tudominio.com']
8 CSRF_TRUSTED_ORIGINS = ['https://tudominio.com']

```

Listing 26: Configuración de seguridad HTTP para el entorno de producción.

14.5 Control de Acceso por Roles

Rol	Cert. X.509	Puede Firmar	Alcance sobre Expedientes
Administrador	Sí (auto-firmado, CA raíz)	Sí	Todos los expedientes; único que puede eliminar.
Coordinador (5 tipos)	Sí (firmado por Admin)	Sí	Expedientes de su rol; edita, verifica y firma.
Operativo	No	No	Lee expedientes de su rol; crea nuevos registros.
Usuario	No	No	Solo ve expedientes que él mismo creó.
Migrante	No aplica	No	Solo su propio expediente, vía portal público.

Tabla 18: Capacidades criptográficas y alcance de acceso por rol.

15. Guía de Contribución

15.1 Convenciones de Código

- **Lenguaje:** todo el código, comentarios y nombres de variables están en español, excepto las palabras reservadas del lenguaje y los nombres de métodos heredados de Django/Python.
- **Estilo:** se sigue PEP 8 para Python con las siguientes excepciones pragmáticas: líneas de hasta 120 caracteres en archivos de vistas y modelos, y alineación vertical en definiciones de diccionarios y tablas de roles.
- **Nombres de modelos:** sustantivos en español con PascalCase (`Expediente`, `AccesoExpediente`, `SolicitudARCO`, `BitacoraEvento`).
- **Nombres de funciones criptográficas:** verbos en español con snake_case (`cifrar_datos_sin_rsa`, `descifrar_llave_con_password`, `derivar_clave_folio`, `generar_par_llaves`).
- **Decoradores:** prefijo descriptivo (`@rol_requerido`, `@firma_requerida`, `@sesion_migrante_requerida`).

15.2 Flujo de Trabajo (Git)

- Cada *feature branch* se crea desde `develop`.
- Los commits siguen el formato: `[modulo] Descripcion breve del cambio` (e.g., `[cripto] Agregar validacion de modulus al ingresar firma`).

- Pull requests requieren revisión de al menos un miembro del equipo antes del merge.

15.3 Cómo Agregar Funcionalidades

Para agregar una nueva funcionalidad al sistema:

1. Crear un branch desde `develop` con nombre descriptivo (`feature/nueva-funcionalidad`).
2. Si la funcionalidad involucra datos sensibles, implementar el cifrado usando las funciones existentes en `cripto/crypto.py`. No crear nuevas primitivas criptográficas sin justificación.
3. Si la funcionalidad modifica expedientes o datos de usuario, registrar el evento correspondiente en `BitacoraEvento`.
4. Si la funcionalidad requiere un nuevo rol o permiso, agregarlo en `usuarios/roles.py` y en el management command `inicializar_llaves_rol.py`.
5. Documentar el caso de prueba siguiendo la plantilla en `test_cases/_TEMPLATE.md`.
6. Ejecutar `python manage.py makemigrations` y `python manage.py migrate` antes de commitear.
7. Crear pull request hacia `develop` con descripción del cambio y evidencia de pruebas.

16. Problemas Conocidos

16.1 Limitaciones del Hosting (HostGator)

- **Sin control de procesos:** los planes de hosting compartido de HostGator no permiten ejecutar Gunicorn como daemon. La alternativa es usar Phusion Passenger (si está disponible en el cPanel) o FastCGI con `flup6`.
- **Sin acceso root:** no es posible instalar dependencias del sistema operativo (como `libffi-dev` o `libssl-dev`). Las dependencias Python deben compilarse contra las versiones del sistema disponibles.
- **Límites de memoria:** los planes básicos limitan la memoria por proceso. Scrypt con $N = 16384$ requiere aproximadamente 16 MB de RAM por derivación; esto puede ser un problema bajo carga concurrente alta.
- **Sin WebSocket:** no se soportan conexiones persistentes. Todas las actualizaciones requieren *refresh* del navegador.

16.2 Limitaciones Técnicas del Sistema

- **Sin autenticación multifactor (MFA):** el sistema actualmente depende exclusivamente de contraseña + llave de firma. No implementa TOTP, SMS ni hardware tokens.
- **Sin recuperación de .key:** si un usuario pierde su archivo `.key` o su passphrase, la única solución es que un Administrador regenere su identidad criptográfica completa, lo que invalida firmas anteriores.

- **Archivos de prueba vacíos:** los archivos `tests.py` de todas las aplicaciones Django están vacíos. La validación se realiza mediante los 130 casos de prueba manuales documentados.
- **Sin configuración LOGGING explícita:** en producción, los errores técnicos dependen del logging por defecto de Django (`stderr`). Se recomienda agregar un handler de archivo.
- **Sin vistas de error personalizadas:** no existen templates `404.html` ni `500.html`. Django muestra sus páginas de error genéricas en producción.

17. FAQ Técnica

17.1 Problemas Comunes al Levantar el Proyecto

P: Aparece ImportError: No module named 'Crypto' al iniciar el servidor.

R: Verificar que `pycryptodome` esté instalado y que no exista el paquete `pycrypto` en el entorno virtual. Ambos no pueden coexistir. Solución: `pip uninstall pycrypto && pip install pycryptodome`.

P: La base de datos no conecta y aparece OperationalError: (2003, 'Can't connect to MySQL server').

R: Verificar que MySQL esté corriendo (`sudo systemctl status mysql`), que las credenciales en `.env` sean correctas, y que la base de datos `casa_monarca` exista. Crear con:

```
mysql -u root -p -e "CREATE DATABASE casa_monarca CHARACTER SET utf8mb4;"
```

P: Al ejecutar inicializar_llaves_rol aparece un error de integridad (duplicados).

R: El comando solo debe ejecutarse una vez sobre una base de datos limpia. Si ya existen registros en `LlaveRol`, el comando fallará por la restricción de unicidad en el campo `rol`. Solución: eliminar los registros existentes o ejecutar sobre una BD recién migrada.

P: Los campos cifrados aparecen como texto ilegible en Django Admin.

R: Esto es comportamiento esperado. Los campos `EncryptedTextField` y `EncryptedCharField` se cifran transparentemente en la capa ORM; en el Django Admin se muestra el texto plano porque se accede a través del ORM. Si se inspecciona directamente la tabla MySQL (con un cliente SQL), se verá el ciphertext en Base64.

17.2 Errores Típicos de Desarrollo

P: Cambié el SECRET_KEY y ahora todos los campos cifrados son ilegibles.

R: El `SECRET_KEY` de Django se usa como raíz para el cifrado transparente de BD (`SHA-256(SECRET_KEY)` → clave AES). Cambiarlo invalida todos los campos `EncryptedCharField`, `EncryptedTextField` y `EncryptedTextFieldExpediente`. No existe mecanismo de migración automática. Solución: restaurar el `SECRET_KEY` original o reconstruir la base de datos desde cero.

P: La firma digital falla con “Llave de firma incorrecta” aunque la passphrase es correcta.

R: Verificar que el archivo `.key` corresponda al usuario que lo está cargando. El sistema valida que el modulus de la llave privada del `.key` coincida con el modulus del certificado X.509 registrado

para ese usuario. Si el certificado fue regenerado después de emitir el `.key`, el archivo anterior ya no es válido.

P: El middleware de certificados expulsa al usuario inmediatamente después del login.

R: El `CertificadoExpiracionMiddleware` verifica `fecha_expiracion_certificado < now()`. Si el certificado venció, el middleware fuerza logout en cada request. Solución: un Administrador (con certificado vigente) debe regenerar la identidad del usuario afectado desde el Panel de Administración.

P: Un Operativo no puede descifrar expedientes aunque tiene el rol correcto.

R: Verificar que exista un registro en `AccesoLlaveRol` para ese usuario y el rol Operativo, y que la sesión tenga `_llaves_rol_cache['Operativo']` poblado. Si la llave no se descifró durante el login (por contraseña incorrecta al momento de cifrar, o salt corrupto), la sesión criptográfica estará vacía. Solución: un Administrador debe hacer reset de contraseña para regenerar las llaves y redistribuir accesos.

P: El portal del migrante devuelve “Folio o nombre incorrecto” aunque los datos son correctos.

R: La derivación Scrypt es case-sensitive en el nombre normalizado (`strip().lower()`). Verificar que el nombre completo coincida exactamente con el registrado (primer nombre + primer apellido + segundo apellido). También verificar que no se haya alcanzado el rate-limit (5 intentos / 15 min por IP); esperar 15 minutos o probar desde otra IP.

18. Referencias

- [1] Barker, E., et al. (2019). *Recommendation for Key Management: Part 1 — General*. NIST Special Publication 800-57. <https://csrc.nist.gov/publications/detail/sp/800-57-part-1/rev-5/final>
- [2] Ramió Aguirre, J. (2006). *Libro electrónico de seguridad informática y criptografía*. Universidad Politécnica de Madrid.
- [3] Shannon, C. E. (2001). *A Mathematical Theory of Communication*. ACM SIGMOBILE Mobile Computing and Communications Review, 5(1), 3–55.
- [4] Django Software Foundation (2024). *Django Documentation*. <https://docs.djangoproject.com/en/4.2/>
- [5] PyCryptodome (2024). *PyCryptodome Documentation*. <https://pycryptodome.readthedocs.io/>
- [6] Python Cryptographic Authority (2024). *cryptography library*. <https://cryptography.io/en/latest/>
- [7] PyMySQL (2024). *PyMySQL Documentation*. <https://pymysql.readthedocs.io/>
- [8] RFC 5280 (2008). *Internet X.509 Public Key Infrastructure Certificate and CRL Profile*. <https://tools.ietf.org/html/rfc5280>
- [9] FIPS 197 (2001). *Advanced Encryption Standard (AES)*. National Institute of Standards and Technology. <https://csrc.nist.gov/publications/detail/fips/197/final>

- [10] *Ley Federal de Protección de Datos Personales en Posesión de los Particulares* (LFPDPPP).
Diario Oficial de la Federación, México, 5 de julio de 2010.